Standard Operating Procedure (SOP): Arctos Robotic Arm CAN Bus & ROS 2 Integration

Robotics Engineering Laboratory Guide

August 2026

Contents

1 Introduction & Core Concepts

Welcome to the Arctos Robotic Arm Integration Guide! This Standard Operating Procedure (SOP) will teach you how to wire, program, and control an industrial-style robotic arm using a communication framework called a **CAN Bus** (Controller Area Network) wrapped inside **ROS 2** (Robot Operating System).

Before writing code, there are two engineering concepts you must understand to prevent damaging the hardware:

1.1 The Power of Mechanical Advantage (Gearboxes)

The motors inside our robot do not drive the arm joints directly. Instead, they spin through a heavy reduction gearbox with a **67.82:1 ratio**.

- This means the inner motor shaft must spin completely around **67.82 times** just to rotate the physical outer robot arm joint by **1 single turn** (360°).
- Because of this massive scaling, commanding the motor to spin a small distance results in tiny, highly precise shifts on the arm axis.

1.2 Closed-Loop Control & Artificial Stalls

Our motors use MKS Servo42D closed-loop controllers. Traditional stepper motors blindly take steps without knowing if they actually moved. Our closed-loop motors use a magnetic sensor on the back of the shaft to read exact positioning.

The Safety Catch: If the robot arm hits an obstruction or the timing belt is pulled too tight, the motor shaft will fall slightly behind its electronic target. The driver board will instantly trigger a **Tracking Error Protection Stall**, locking the motor completely down to prevent drawing excessive current or breaking parts. **If your motor stops responding completely and goes limp, you must power-cycle its main 12V/24V power supply to clear the safety latch.**

2 Safety First! Emergency Procedures

Robotic arms possess immense torque and can easily pinch fingers or crush components if commanded incorrectly.

1. **Keep Your Distance:** Never place your hands near the belts, pulleys, or joint linkages while the main power supply is active.
2. **The Kill Commands:** Keep a separate terminal window open at all times running `candump -tz can0` to watch live network traffic. If the arm moves unexpectedly, prepare to type or execute the following emergency stop frame immediately:

```
# Instant Emergency Stop (Cuts all holding torque) -- Joint B, CAN ID 0x05
cansend can0 005#F7FC

# Explicit Motor Coil Disable -- Joint B, CAN ID 0x05
cansend can0 005#F300F8
```

Listing 1: Emergency Stop Over CAN Command Line

Caution: Every frame needs its checksum byte, including the emergency stop

Earlier versions of this document sent `F7` and `F300` with no checksum byte at all. Reading `motor_driver.cpp`'s `stopMotor()` and `can_protocol.cpp`'s `sendFrame()` directly confirms the real driver **always** appends a checksum – `sendFrame()` has no special case for `EMERGENCY_STOP`. The checksum is `(CAN_ID + command_byte + ... + data_bytes) & 0xFF` as used everywhere else in this document. For Joint B (`0x05`): `0xF7+0x05=0xFC`, so the frame is `F7FC`; for `F3 00` (disable): `0xF3+0x00+0x05=0xF8`, so `F300F8`. **Recompute the checksum for whichever CAN ID you're actually targeting** – don't copy the Joint B byte values above for a different joint.

3 Hardware Map & Network Setup

The communication backbone relies on the **CANable 2 USB Adapter**. This device translates serial data coming from our computer into high-speed physical differential voltage lines (`CAN_H` and `CAN_L`) that daisy-chain from motor to motor down the body of the robot chassis.

3.1 The Joint Architecture Matrix

Each joint on the network operates using a specific identifier (CAN ID). When sending commands over the bus, you must target these hex values exactly:

Robot Joint	Physical Role
Joint X	Base/waist rotation (first joint from `base_link`)
Joint Y	(role not confirmed in repo docs)
Joint Z	(role not confirmed in repo docs)
Joint A	(role not confirmed in repo docs)
Joint B	Wrist Rotation
Joint C	Elbow Pitch (per project convention; note this is the *last* joint before the end effector in the UI

Table 1: Arctos Arm Network Node Mappings, per `motor_id` in `arctos_description/config/arctos_controller.yaml` – the real 6-joint arm configuration. **Confirm every value in this table empirically with `can_id_scan.py` before trusting it** – Joint C's CAN ID was listed as `0x03` in earlier drafts of this document and in other guide documents in this project (that value is actually Joint Z's ID in a separate, simplified 3-joint test-rig config, `arctos_moveit_base_xyz/config/ros2_controllers.yaml`), and Joint X's DIP switches turned out not to map to any CAN ID at all once it was discovered that driver was the RS485 hardware variant (Section 5.1.1).

3.2 Bringing Up the SocketCAN Network (First Time SOP)

Every time you plug the CANable adapter into your Ubuntu computer, you must bridge the serial hardware interface to the Linux networking core. Follow these steps exactly:

```
# 1. Verify your computer sees the USB hardware device
lsusb | grep -i canable
# Expected: ID 16d0:117e CANable2

# 2. Identify the active serial port handle
ls -l /dev/ttyACM*
# Note down if it returns /dev/ttyACM0 or /dev/ttyACM1

# 3. Clean out old dead connections
sudo pkill slcand
sudo ip link delete can0 2>/dev/null

# 4. Bind the adapter to the Linux socket layers at 500kbit/s (-s6)
sudo slcand -o -c -s6 /dev/ttyACM0 can0
sudo ip link set can0 up

# 5. Check to confirm the network state is ACTIVE
ip -details -statistics link show can0
```

```
# Look for: <NOARP,UP,LOWER_UP> and state ERROR-ACTIVE
```

Listing 2: Linux Terminal CAN Interface Setup

3.3 Automated Setup Script

`scripts/setup_canable.sh` in the `ros2_arctos` repository automates the five steps above. **If you pulled this repo before August 2026, re-pull it first**: the script previously attached the adapter with `slcand -s3` (100 kbit/s) and then tried to override the bitrate with `ip link set can0 type can bitrate 500000` – a netlink attribute that native slcan interfaces do not support, so the override silently failed and the bus was left running at the wrong speed. It now attaches directly at `-s6` (500 kbit/s), matching the manual steps above.

```
cd ~/arctos_ws/src/ros2_arctos
chmod +x scripts/setup_canable.sh
sudo ./scripts/setup_canable.sh
```

Listing 3: Running the Automated Setup Script

4 Protocol Errata: Reconciling This Project's Debugging Documents

This workspace accumulated several other debugging documents alongside this one – `arctos_CAN_ROS2_SOP_updated`, `arctos_joint_b_guide.tex`, and `arctos_motor_can_cheat_sheet_updated.md` – written at different points while reverse-engineering the same MKS CAN protocol. They do not fully agree with each other. Rather than silently pick one, this section cross-checks the disagreements directly against the real source (`motor_types.hpp`, `can_protocol.cpp`, `motor_driver.cpp`) and records the resolution here, since that source is the actual ground truth for what the C++ ROS 2 driver will do.

Caution: Encoder read opcode: use `0x31`, not `0x30`

`arctos_joint_b_guide.tex` (the older of the two documents) standardizes on `0x30` as "the confirmed encoder read command," describing a carry+value response format, and treats `0x31` as unconfirmed. This does not hold up: `motor_types.hpp` defines `READ_ENCODER = 0x31` and no `0x30` command at all – the only place `0x30` appears anywhere in `arctos_motor_driver`'s source is as raw test-fixture bytes for an unrelated velocity-decoding unit test, not a CAN command opcode. `arctos_CAN_ROS2_SOP_updated.tex` reaches the correct conclusion (`0x31`, decoded via the 48-bit `decodeInt48()` format used throughout this document) by citing the same source files. **Use `0x31` with the 48-bit decode, as every script in this package already does.** Consequently, the old `mks_driver.py`'s `read_absolute_encoder()` (in both `arctos_can` and the early `arctos_can_control` draft), which sends `0x30`, is querying a command the real driver does not define and should not be trusted or built upon.

Caution: Checksum required on every frame, including disable and emergency stop

Several manually-run `cansend` examples across these documents (and in earlier drafts of this SOP, now fixed in Section 2) omit the checksum byte on `F3 00` (disable) and `F7` (emergency

stop). `can_protocol.cpp`'s `sendFrame()` has no special case for any command – it always appends `(motor_id + sum(data_bytes)) & 0xFF` as the last byte. Always compute and include it, as every script in this package's `send_frame()`/`checksum()` pair already does.

Caution: Working-mode mismatch can look like a communication failure

Both newly-added documents independently flag something not previously captured in this SOP: `motor_types.hpp` defines a six-way `MotorMode` enum (`CR_OPEN=0`, `CR_CLOSE=1`, `CR_vFOC=2`, `SR_OPEN=3`, `SR_CLOSE=4`, `SR_vFOC=5`), set via `SET_WORKING_MODE (0x82)`, and the ROS 2 driver defaults to assuming `working_mode=2` (`CR_vFOC`). If the physical driver's own panel/config is set to a different mode than whatever a script assumes, the controller can accept and checksum-acknowledge a motion frame (`0xF5`/`0xF4`) **without the shaft actually rotating** – which reads exactly like a wiring or CAN-ID problem but isn't one. If Step 3 of Section 5 sends a checksum-acknowledged move and the encoder genuinely does not change afterward (not just a small/ambiguous change – see the next caution), check the driver's working-mode setting before assuming the CAN link or the frame format is at fault.

Caution: The `-813 pulses/degree` constant and any `0x30`-based safety limit are not trustworthy

Consistent with Section 5.1.1's and this section's other findings: the `joint_profiles` placeholder in `arctos_can/mks_driver.py` (`pulses_per_degree = -813.0`) does not match the source-verified ≈3086.56 counts/degree figure for the 67.82:1-geared joints (off by roughly 3.8×), and its accompanying "safety workspace cage" estimates the limit using a different multiplier (`-0.947`) than what is actually transmitted, so it cannot reliably predict or block an unsafe target. Do not reuse this constant or that limit-checking code for Joint C or any other 67.82:1 joint without first reconciling it against the gear-ratio math confirmed in this document.

---

5 Bringing Up an Uncalibrated Joint (One at a Time)

This project's actual practice has been to wire up and bring up **one joint at a time** – first Joint X, then (after that driver turned out to be the wrong hardware variant, Section 5.1.1) Joint C – rather than trust the full arm's wiring and configuration all at once. This section is written as a repeatable procedure for whichever joint is currently the only one wired to the bus: get trustworthy *information* out of it first, and only reach actual *motion* as a small, explicitly-confirmed last step. Table 2 gives the confirmed parameters for each joint bench-tested so far; substitute the relevant row into the generic commands below.

Neither CAN ID in the table above should be trusted without running Step 1 – Joint X's DIP switches (2 and 3 ON) turned out not to map to any working CAN ID at all once it was discovered that driver was the RS485 hardware variant, and Joint C's ID has disagreed across this project's own documents (`0x03` in an earlier hardware-map table and in other guide documents found in this workspace, versus `motor_id: 6` i.e. `0x06` in the authoritative `arctos_controller.yaml` – the `0x03` value actually belongs to a different joint, `Z_joint`, in a separate simplified 3-joint test-rig config).

Joint	CAN ID
X	`0x01` (per YAML; driver was actually RS485, Sec. 5.1.1)
C	`0x06` per `arctos_controller.yaml`, `0x03` per older project docs – **neither correct; confirmed** `0x01`

Table 2: Joint instances bench-tested with this procedure so far. Both `inverted=true`, both `requires_homing=false` per `arctos_controller.yaml`. Encoder resolution is 16384 counts/motor-revolution for both.

Do not use the placeholder values in the `joint_profiles` dict of `arctos_can/mks_driver.py` either, which were copy-pasted from Joint B's calibration and are not valid for any other joint.

5.1 Step 1 – Confirm the Joint's Real CAN ID

Run the discovery script below. It only sends the read-only `0x31 READ_ENCODER` query (the same opcode the real C++ driver in `arctos_motor_driver` uses) to a range of candidate IDs and reports who answers. It never enables coils and cannot move the arm.

```
cd ~/arctos_ws
source install/setup.bash
python3 src/arctos_can_control/arctos_can_control/can_id_scan.py --channel can0 --
    ids 1-16
```

Listing 4: Confirming a Joint's CAN Address

Exactly one ID should respond, since only one joint should be wired at a time. Use that confirmed value – not the table above by assumption, and not a guess from the DIP switches – for every command in the rest of this section.

5.1.1 Troubleshooting: Zero Responses AND Zero Bus Errors

If the scan above (and a full baud-rate sweep across every standard rate, DIP switches aside) comes back completely silent **with the link statistics also showing zero bus-errors** (`ip -details -statistics link show can0`), stop tuning baud rates and DIP switches and check the driver's part number first. A real CAN node at the wrong bitrate still generates bus errors, because it is at least attempting to decode CAN framing and losing arbitration/ACK. Total silence with zero errors, despite confirmed 12–24V driver power and confirmed CAN_H/CAN_L continuity and termination, is the signature of a driver that has **no CAN transceiver at all** – most commonly because it is the **RS485 variant** of the MKS SERVO42D/57D rather than the CAN variant. RS485 and CAN both use a differential pair, but RS485 here carries MKS's own async serial command protocol, not CAN framing; a real CAN controller and an RS485 transceiver cannot talk to each other over the same wires no matter what bitrate or address is configured. If this happens, physically confirm the board's part number/silkscreen before spending more time on the software side, and if it is the RS485 variant, either exchange it for the CAN variant (keeps this entire document, the shared CAN daisy-chain, and every script in this package working unchanged) or treat that joint as a separate RS485 subsystem requiring its own USB-RS485 adapter and a driver written against MKS's RS485 protocol document, which is a different frame format and checksum from everything described here and is out of scope for this SOP. **This is exactly what happened with Joint X** – the driver installed there was the RS485 variant, which is why Joint C is now the joint under test instead.

5.2 Step 2 – Read-Only Reliability Test

Before commanding any motion, verify the CAN link to this joint is actually trustworthy: response rate, checksum integrity, and encoder jitter while the joint sits still. `joint_reliability_test.py` sends only `0x31` (encoder), `0x39` (error state), and `0x3A` (enable state) – it never sends `0xF3` (enable) or a position command, so coils are never energized and the motor cannot move as a result of running it.

```
# Joint C example -- substitute the confirmed CAN ID and correct gear ratio
# from Table 3 for whichever joint is actually wired up.
python3 src/arctos_can_control/arctos_can_control/joint_reliability_test.py \
    --channel can0 --can-id 0x01 --gear-ratio 67.82 --samples 200
```

Listing 5: Joint Reliability / Telemetry Test

The script reports a response rate, checksum failure count, round-trip latency, and stationary encoder jitter, then prints a plain-language verdict on whether the link is clean enough to move on to Step 3. Do not proceed to Step 3 until response rate is above 95% with zero checksum failures.

5.3 Step 3 – Supervised First-Motion Test

Run this yourself, standing next to the arm with a hand on the power switch. Do not run it unattended or from a remote session. If this is a joint's first-ever motion test, its coils have never been energized before and it has no calibrated travel limits yet, so nothing in software stops it from driving into a mechanical hard stop if a sign or scale assumption below turns out wrong.

```
# In a second terminal, keep this running the whole time:
candump -tz can0

# In your main terminal (Joint C example):
python3 src/arctos_can_control/arctos_can_control/joint_micro_move_test.py \
    --can-id 0x01 --gear-ratio 67.82
```

Listing 6: Supervised Joint Tiny-Move Test

The script prints the starting encoder position, then requires you to type `go` before it enables coils and commands a small (˜0.1–0.3° of joint travel) absolute move, printing live position every half second so you can watch for stall or runaway behavior. Ctrl+C at any point sends an immediate, checksummed `0xF7` emergency stop – the script prints the exact checksummed frame for whichever CAN ID you passed it at startup, so you can also trigger it manually from another terminal at any time, e.g. for Joint C (confirmed `0x01`): `0xF7+0x01=0xF8`:

```
cansend can0 001#F7F8
```

Listing 7: Emergency Stop for Joint C

5.3.1 Troubleshooting: Enable Acknowledged, But No Holding Torque

Joint C's actual bench test hit this: `can_id_scan.py` found it at `0x01` (not `0x06` per `arctos_controller.yaml`, not `0x03` per older docs), Step 2's reliability test came back perfect (100% response rate, 0 checksum failures, 0 encoder jitter), and `ENABLE_MOTOR` was checksum-accepted with `READ_ENABLE_STATE (0x3A)` confirming a flip from disabled to enabled – but the shaft still spun completely freely by hand, with no holding torque at all, and a commanded `0xF5` move produced zero encoder change over several seconds.

The real `arctos_hardware_interface` source (`arctos_interface.cpp`'s `setupMotorParameters()`) sends `SET_WORKING_MODE (0x82)` with `MotorMode::SR_vFOC` (value `5`) to every joint before ever enabling it – something none of the raw Python scripts in this document had been doing. Sending that first (`cansend can0 001#820588` for CAN ID `0x01`: $0x82 + 0x05 + 0x01 = 0x88$) did make `READ_ENABLE_STATE` flip from `0` to `1` on the next enable – but the shaft was still free-spinning, meaning that CAN-reported flag does not by itself confirm the coils are actually driven. `motor_types.hpp` also defines `SET_CURRENT (0x83)` (payload: 16-bit little-endian mA) with a source default of `working_current{1600}`, and its setter is likewise never actually called anywhere in the real ROS 2 stack (the call site in `setupMotorParameters()` is commented out) – so it depends entirely on whatever the panel has stored. Explicitly setting it to this motor family's documented rated current (1200 mA) before enabling made no difference either.

At that point, two CAN-configurable causes had been ruled out with real test data, which shifts the likely cause to something CAN commands cannot fix remotely:

- **Motor phase wiring** – confirm all four phase wires are firmly seated between the driver's motor output terminals and the motor itself. A loose or disconnected phase wire lets the driver's internal state machine believe it is enabled and driving current, while delivering no actual torque.
- **Main motor power specifically to this driver** – confirm the 12–24V motor power rail (not just logic/CAN power) is actually connected to *this* driver board. A driver can respond to CAN reads/writes on logic power alone while having no motor-power rail connected at all.
- `SET_ENABLE_SETTINGS (0x85)` – defined in `motor_types.hpp` but its payload format is not documented anywhere in this project (no guide document, no source comment). Do not guess-write to this opcode; if the driver has its own physical screen/panel, check its enable-pin/enable-mode setting there directly instead.

5.4 Step 4 – Read-Only ROS 2 Telemetry

Once Steps 1–2 pass, `joint_state_publisher` exposes live joint position on `/joint_states` without any ability to command motion – it has no command subscriber by design, so it is safe to leave running continuously while you work on calibration.

```
cd ~/arctos_ws
colcon build --packages-select arctos_can_control
source install/setup.bash

# Terminal 1: telemetry-only node (Joint C example)
ros2 run arctos_can_control joint_state_publisher --ros-args \
    -p joint_name:=C_joint -p can_id:=6 -p gear_ratio:=67.82

# Terminal 2: watch live position
ros2 topic echo /joint_states
```

Listing 8: Joint ROS 2 Telemetry Node

Only after Step 3 has been repeated a few times with consistent, expected direction and magnitude – and real travel limits have been established, e.g. by wiring and testing the hall-effect home sensor per the `ros2_arctos` README – should a joint move on to closed-loop position commands from a higher-level controller.

6 Session Log: Joint C Bring-Up Attempt (2026-08-25)

Picking this up again: Joint C is wired, powered, and communicating over CAN, but **does not produce holding torque under any configuration tried today**. This section records exactly what was tried and ruled out, so the next session doesn't repeat it.

6.1 Confirmed working

- CAN link is solid: Step 2's reliability test passed cleanly (100% response rate, 0 checksum failures, 0 encoder jitter across 200 samples).
- CAN termination jumper near the CAN_H/CAN_L terminals was confirmed present and left installed – correct, since this driver is currently one of only two nodes on the bus (CANable and this driver).
- Main motor power measured at 24.9 V DC at the driver – normal and expected for this driver family (not the 249.6 V initially misreported).
- All four motor phase wires confirmed firmly seated; the driver-to-motor connector was additionally unplugged and re-seated mid-session. Neither changed the outcome.
- The joint was reportedly calibrated previously. Calibration (`0x80 CALIBRATE`) was **not** re-run today because the motor is currently coupled to its gear train and cannot be decoupled right now – `motor_driver.cpp` explicitly warns calibration should be done unloaded, so re-running it under load was judged higher-risk than leaving it as-is.

6.2 CAN ID: moved during the session – always re-scan, never assume

`can_id_scan.py` first found Joint C responding at `0x01` (matching neither `arctos_controller.yaml`'s `0x06` nor the older docs' `0x03`). Partway through the session the CAN ID was changed on the driver's own panel to `0x06`, confirmed again by re-scanning – **do not assume either value going forward; run `can_id_scan.py` at the start of the next session before anything else.**

6.3 Ruled out: every CAN/panel-configurable setting tried so far

All of the following were tested (via direct CAN commands and/or the driver's own physical panel) with **no change in outcome** – `READ_ENABLE_STATE (0x3A)` sometimes read back `1` (enabled) and sometimes `0` depending on mode, but in every single case the shaft span completely freely by hand with no detectable holding torque:

- **Working mode**: `SR_vFOC` (value 5, set both via `0x82` over CAN and later on the panel directly), `CR_FOC`/`CR_vFOC` (value 2, set on the panel), and `CR_CLOSE` (value 1, set on the panel – this is what Joint B's panel is documented to use).
- **Working current**: the panel's stored default of 3000 mA (roughly 2.5× this motor's documented 1.2 A rating – flagged and corrected), and 1200–1600 mA (matching the documented rated current), both via `0x83` over CAN and via the panel.
- **CAN ID**: `0x01` and `0x06` (see above).

Important corroborating observation: after several enable/hold cycles at up to 3000 mA, **the motor showed no detectable warmth at all**. If current were actually reaching the coils, even without producing net rotation, resistive (I^2R) heating should be noticeable fairly quickly at that current. Its complete absence across every trial points away from a settings/calibration problem and toward an open or broken path between the driver's motor-output stage and the coils – something no CAN command can fix.

6.4 Not yet tried / next session

- **Phase resistance check**: with power off, measure resistance across each of the two motor phase-wire pairs directly at the driver-to-motor connector (a healthy phase for this motor class should read roughly 1–5 Ω; open/infinite indicates a broken phase). Not done today – no multimeter was available.
- **Component swap test**: temporarily connect Joint B's known-working motor to Joint C's driver (or Joint C's motor to Joint B's driver) to isolate whether the fault is in the driver or in the motor/cable. Not attempted today.
- **Live comparison against Joint B**: the written guide documents describing Joint B's setup (`arctos_CAN_ROS2_SOP_updated.tex`, `arctos_joint_b_guide.tex`, the cheat sheet) all still describe Joint B's own motion as unconfirmed as of Aug 19–20, even though Joint B has since been confirmed to move – whatever actually fixed it isn't recorded in any document in this workspace. Check Joint B's driver's *actual current* panel settings directly (working mode, current, CANRSP) and compare against whatever Joint C ends up needing, rather than trusting the stale written docs.
- `SET_ENABLE_SETTINGS (0x85)` exists in `motor_types.hpp` but its payload format is undocumented anywhere in this project and was deliberately not guessed at. If the panel exposes an equivalent enable-pin/enable-source setting, check it there instead.

7 Session Log: Joint B Diagnostics & Large-Move Test (2026-08-27)

Two days after the Joint C session above, Joint C's own troubleshooting was paused (multimeter phase-resistance check and driver/motor swap test still pending – see Section 6), and Joint B was connected and powered instead to use as a known-working reference.

7.1 Joint B: confirmed healthy, with real holding torque

`can_id_scan.py` confirmed Joint B at `0x05` (matching every document, unlike Joint C's repeated mismatches). `joint_reliability_test.py` came back clean: 100% response rate, 0 checksum failures, 1 raw count of jitter (noise floor) across 200 samples. Critically, Joint B's coils were found already enabled (`coils_enabled=True` before this session sent anything), and **the shaft genuinely resisted turning by hand** – real holding torque, unlike every attempt on Joint C.

7.2 Joint B's verified-working panel configuration

There is no `READ_WORKING_MODE` or `READ_CURRENT` opcode anywhere in `motor_types.hpp` – these can only be read from the driver's own physical panel, not queried over CAN. Read directly off

Setting	Joint B's verified-working value
Working mode	`SR_CLOSE` (enum value 4)
Current	1600 mA (matches `motor_types.hpp`'s own `working_current{1600}` default exactly)
CANRSP	Enable

Table 3: Joint B live-verified panel configuration, 2026-08-27.

Joint B's screen while it was confirmed holding torque: **This is a new working mode not yet tried on Joint C** – Joint C testing on 2026-08-25 covered `SR_vFOC` (5), `CR_vFOC`/`CR_FOC` (2), and `CR_CLOSE` (1), but not `SR_CLOSE` (4). Next Joint C session: set `SR_CLOSE` + 1600 mA + confirm CANRSP=Enable, then re-run the hold-enable test.

7.3 20-degree monitored move test on Joint B: real motion, but not reliably reproducible per-command

With Joint B confirmed healthy, a user-requested ∼20° joint move was run as 5 monitored steps of 4° each (Section **??**'s absolute-vs-relative ambiguity for `0xF5` was never resolved, so the incremental approach from Section **??** was used rather than a single blind large command). Starting position was ≈ 0.00, well inside the calibrated envelope (`home_position`=+12.87, `opposite_limit`=−24.23 per `arctos_controller.yaml`).

Result: Step 1 produced real motion in the wrong direction and wrong magnitude (commanded +4, actual −1.84), after which **Steps 2–5 produced essentially no further motion** (each < 0.001) despite each commanding a distinct new absolute target. No protection/stall flag was ever set; final position (−1.85) stayed safely inside the envelope. The incremental/monitored design caught this after a small, safe excursion rather than after a blind full-magnitude commit.

A follow-up test re-sent `ENABLE_MOTOR` immediately before every `0xF5` and added a 3-second pause between steps, using smaller 2 steps. **This produced real motion of very close to the commanded magnitude on 2 of 3 steps** (Step 1: commanded +2.0, actual +1.96; Step 3: commanded +2.0, actual +1.96) – a clear improvement over the first attempt, where only the very first command after enabling produced any real motion at all. **But Step 2 landed nowhere near its own commanded target**, instead landing within ∼13 raw counts of where the joint had been *two steps earlier* – even though the transmitted frame's position bytes were verified correct and distinct from the other two steps. This is not a simple sign-flip or absolute-vs-relative confusion; it looks like a command-queuing or timing race between consecutive `0xF5` frames, where a stale target sometimes wins over the freshly-sent one.

Caution: Do not trust a single `0xF5` command's outcome without reading the encoder back

Across both tests today, magnitude and direction were unpredictable often enough (3 of 8 total commanded steps across both runs landed far from their target) that **every `0xF5` command must be followed by an encoder read to confirm where the joint actually ended up** before commanding anything further, exactly as the incremental procedure in Section **??** already does. No error or shaft-protection flag was set during any of these mismatches – the driver does not flag this condition on its own.

7.4 RELATIVE_POSITION (0xF4): a single move matched commanded magnitude almost exactly

A single 0xF4 command (delta +6173 raw counts, ≈ 2.0°) produced −1.98° of real motion – magnitude within 1% of commanded, direction flipped as expected from B_joint's inverted: true flag (these raw test scripts deliberately do not apply that flip themselves; see the caution in Section 5's tiny-move script). This was the cleanest single-command result of the entire session.

But 3 consecutive 0xF4 commands (same delta, sent with only a single upfront ENABLE_MOTOR), run twice, produced almost no motion on any of the 3 steps both times (< 0.04° per step) – reproducing the same partial-execution pattern seen with 0xF5, just now confirmed on 0xF4 too. This is not an absolute-vs-relative distinction; both commands show the same symptom.

7.5 candump capture: the driver genuinely acknowledges "movement started" – but barely moves

A full unfiltered candump -tz can0 capture during a 3-step 0xF4 sequence resolved an open question: is our own polling code silently discarding relevant frames? No. Every F4 00 32 0A 00 18 1D 6A command received a real, checksummed F4 01 FA response – decoding status=1 against motor_driver.cpp's own convention for ABSOLUTE_POSITION (case 1: // Movement started), **the driver is telling us, correctly, that it started moving, every single time**. No other frames were ever sent besides our own read requests and this one ack. Meanwhile the raw encoder value crept by only ∼115 counts total across all three commands (each requesting 6173) – roughly 2% of one single step.

7.6 90-second and 60-second single-command watches: a genuine two-phase profile, not "just slow"

Watching a single 0xF4 command for much longer than the ∼5s used everywhere else in this project revealed the real shape of the response: a **real, fast burst** of motion in the first few seconds (−737 counts/s at speed=50,accel=10; −428 counts/s at a much gentler speed=15,accel=1), covering the large majority of whatever motion occurs, followed immediately by an **effective stop** – creeping at noise-floor level (∼0.3 counts/s) for the remaining ∼80+ seconds of each test, never approaching the full 6173-count target. Total captured: −2353 counts (38%) at the faster settings, −1301 counts (21%) at the gentler settings – **a much gentler ramp did not fix it, and if anything captured less of the move**, ruling out "acceleration too aggressive for the load" as the primary cause. No 0x3E shaft-protection flag was ever set in either test.

Caution: A real sound was heard at the burst-to-stop transition, on two separate occasions

The user reported hearing something at the point where motion stopped, on two separate test runs. **Physical inspection ruled out an external obstruction**: no visible obstruction, the bare motor shaft (checked before the gearbox) moves freely and does not catch, and **the arm segment itself was confirmed to visibly move** during at least one test (ruling out total gearbox/coupling slippage, since the encoder is mounted on the motor's own shaft *before* the gearbox and would not by itself prove the joint's output side actually moved). The combination points toward the motor **losing synchronization mid-move** (a stepper "chuff"/step-loss event) rather than a hard mechanical limit. **The motor was also reported "quite warm"** after this sequence of tests

at 1600 mA (documented rated current is 1.2 A) – testing was paused for a cool-down rather than continuing to add more energized time on top of that.

7.7 A promising lead: the real production system never sends one far-away target

Checking `arctos_interface.cpp`'s `write()` function directly: it only calls `setJointPosition()` when the commanded position changes by more than `position_tolerance_` – meaning in real operation, a `JointTrajectoryController` continuously publishes a smoothly interpolated, frequently-updated intermediate setpoint (at the controller's 100 Hz `update_rate`), and `write()` forwards each meaningfully-different point as a fresh CAN frame. **Every test in this entire session has instead sent one single far-away target and waited** – the opposite of how this driver has ever actually been exercised by the real stack. The burst-then-stop pattern is consistent with the driver executing some bounded internal motion profile per command and requiring a fresh, nearby target to keep advancing, which one-shot far-away commands never provide. Not yet tested: `joint_streamed_move_test.py` (added this session, not yet run) sends the total move as many small increments in rapid succession to test this directly.

7.8 A second promising lead: `POSITION_CONTROL (0xFD)`, not `0xF4/0xF5`, is what the project's own working jog code actually uses

`POSITION_CONTROL (0xFD)` is defined in `motor_types.hpp` but **never referenced anywhere in `motor_driver.cpp`** – the C++ ROS 2 driver only ever uses `0xF5`. It *is*, however, the command used by `scripts/set_zero_position.py`'s `relative_motion_by_pulses()`, called repeatedly in a loop from `find_home_position()`, `find_opposite_limit()`, and `manual_move_joint()` – i.e., exactly the repeated-small-jog pattern this session has been struggling to get right with `0xF4/0xF5`, and part of the one documented setup flow (`--init` in the `ros2_arctos` README) that this project actually relies on working. **Its frame layout is structurally different**: a direction bit (`0x80/0x00`) is OR'd into the top of the speed-high byte, and its position field is in **microstep pulses** (200 full steps/rev × 16 microsteps/step = 3200 pulses/motor-rev) rather than the 16384-count/rev encoder scale every other script in this document uses for `0xF4/0xF5`. Not yet tested: `joint_position_control_test.py` (added this session, matches `set_zero_position.py`'s exact byte-packing) is ready to try next session.

7.9 New reusable scripts added this session

- `joint_stepped_move_test.py` – generalized N-step monitored move (absolute or relative, `--mode`), safety-band checked against calibrated `home_position/opposite_limit`, optional `--re-enable-each-step`.
- `joint_streamed_move_test.py` – sends a total move as many rapid small `0xF4` increments instead of one command, to test the trajectory-streaming hypothesis above. Not yet run.
- `joint_position_control_test.py` – `0xFD`-based jog test matching `set_zero_position.py`'s exact pulse-scale and direction-bit convention. Not yet run.

7.10 Characterizing (not resolving) 0xFD: a real, deterministic success signal with a still-unknown trigger

joint_position_control_test.py (0xFD) revealed a protocol detail this whole investigation had been missing: **the driver sends an unsolicited two-frame status sequence per command** – an immediate FD 01 (status=1, "movement started", matching motor_driver.cpp's own convention) followed, sometimes, by a second FD 02 (status=2, almost certainly "movement complete") arriving up to ∼425 ms later. **Whether that second frame arrives correlates perfectly with whether the commanded magnitude is actually achieved** – confirmed by directly matching a raw candump capture against per-step results across an 8-step test: both steps that received FD 02 moved within 1% of their commanded 1°; all six steps that received only FD 01 moved less than 0.04° each, several essentially zero.

An initial 3-step test (re-enabling before every command) looked like a full resolution: individual steps of +0.025°, +1.974°, +0.998° summed to +2.997° against a 3.0° total commanded (99.9%), suggesting an incomplete command's shortfall carries forward into the next one. **This did not hold up when confirmed over a longer 8-step sequence**: steps 1–2 moved correctly (matching FD 02 frames confirmed via candump), step 3 partially, and steps 4–8 essentially stopped entirely (each receiving only FD 01) despite an identical fresh-enable-before-each-command pattern – total movement was 32% of commanded, not 99.9%. A follow-up run logging READ_ERROR (0x39) and READ_IO (0x34) (the latter never queried anywhere else in this project) at fine resolution around a fresh sequence found **the very first command of that run failed too** (only FD 01, negligible motion) – contradicting even the narrower "the first 1–2 commands always succeed" pattern.

Neither additional status opcode explained the difference: READ_IO (0x34) stayed completely constant ([52, 1, 58]) across every query in that run regardless of success or failure, and READ_ERROR (0x39)'s trailing bytes changed on nearly every single query whether or not real motion occurred – behavior more consistent with a free-running counter than a meaningful error/stall code. READ_SHAFT_PROTECTION_STATE (0x3E) never once left its baseline value across any test this session, successful or not.

A retry-on-FD-02 mechanism (joint_position_control_test.py, up to 4 retries/step, always re-enabling first) was then built and tested: in one run, **all 15 attempts across 3 steps got only FD 01, never a single** FD 02 – yet the joint still accumulated real, if uneven, motion across those retries. This broke the working theory that FD 02 cleanly gates real motion, and pointed toward something more fundamental than a CAN-protocol quirk.

7.11 ENABLE_SHAFT_PROTECTION (0x88): a promising lead that turned out to be a false-positive trap

The user reported that manually assisting the joint by hand let a stuck move complete – direct physical evidence pointing at genuine mechanical resistance. This led to checking whether ENABLE_SHAFT_PROTECTION (0x88) – the command that actually turns on the "Tracking Error Protection Stall" feature described all the way back in Section 1 of this document – had ever been sent. **It had not, anywhere in this project**: not in any test script this session, not in the real C++ arctos_hardware_interface, not in set_zero_position.py. Without it enabled, READ_SHAFT_PROTECTION_STATE (0x3E) reads its baseline value regardless of whether a real stall is occurring.

Sending 0x88 0x01 once, then repeating a 1° move, appeared to confirm it immediately: **the very first move afterward tripped the flag** (status 0 → 1), with the joint having moved only 0.0017° before the driver locked itself down (requiring a power-cycle of its main supply to clear, per

Section 1). Raising current to 2000 mA and separately trying a much gentler speed (15 vs. 50) both still tripped *instantly* on the very first command at that setting – a pattern insensitive to speed is itself a clue against a straightforward "not enough dynamic torque" stall, since that failure mode should get *better*, not stay identically instant, as speed decreases.

The user's own comparison test settled it: after checking (this time correctly) that this NEMA17's rated current is 1.2 A – not 2.8 A, that number belongs to the unrelated NEMA23 originally installed on Joint X – the exact same command that had just tripped at 2000 mA and speed 15 was re-sent **without** `0x88` enabled. **It moved almost perfectly**: −1.0008° against a −1.0° commanded target (99.9%), with both `FD 01` and `FD 02` received. **This refutes "confirmed genuine stall" as this document previously claimed**: the identical command, current, and speed produced a clean, correct move the moment `0x88` was left off. `ENABLE_SHAFT_PROTECTION` on this driver appears capable of a false-positive trip – possibly reacting to a normal, brief startup transient present at the start of every move – not only a genuine sustained one.

Caution: `0x88` is off by default in this package's scripts – treat any trip as a lead, not proof

Both `joint_position_control_test.py` and `joint_stepped_move_test.py` now gate `ENABLE_SHAFT_PROTECTI` behind an explicit `--enable-shaft-protection` flag rather than sending it automatically, exactly because of the false-positive result above. If it does trip during real use, cross-check by re-running the identical command with it disabled before concluding a real stall occurred – and remember a trip latches the driver down and costs a power-cycle to clear, so triggering one by default on every session is expensive for a signal that isn't fully trustworthy yet.

7.12 Final resolution: a real, localized mechanical catch, found and fixed by hand

A proper statistical test settled the "incomplete move" question for good. `joint_b_statistical_test.py` (20 reps of ±0.5°, alternating direction so the joint returns near its start each pair, at the correctly-rated 1.2 A, speed 15, `0x88` off) came back **20/20 (100%)**, every rep within 1% of commanded magnitude, no degradation between the first and second half. The same script re-run for **sustained one-direction** motion instead of alternating reproduced the "burst then stop" pattern exactly: 3–4 good reps, then a hard wall – but critically, **starting a fresh sustained run already inside the previously-bad zone failed almost immediately** (1/10), while starting fresh from near 0° took several good reps first. That is the signature of a fixed *position*, not a command count, current level, or session-fatigue effect: alternating tests never accumulate enough one-directional travel to reach it, sustained tests do.

That localized it to roughly −2° to −5° in this joint's raw-command coordinate. Told to check that specific range slowly and deliberately (rather than a quick static check, which had found nothing earlier), the user felt **small catches while rotating through it by hand** – direct physical confirmation. Manually working the gears back and forth through the range (deliberately running the mechanism through the catch to seat/smooth it) was tried, retested, and iterated on the residual narrower sub-range that remained: an initial pass improved a −5° to −1° sustained test from 10% to 70% success; a second pass on the narrower −2° to −1.7° residual brought it to **100% (10/10) in both directions**, every rep 98–101% of commanded magnitude, confirmed across the full previously-affected range.

Caution: The whole "incomplete move" investigation was a real mechanical fault, not a CAN/firmware issue

Every earlier hypothesis this session – `0xF4` vs `0xF5` vs `0xFD`, working mode, current, re-enabling patterns, `FD 02` timing, shaft protection – was chasing symptoms of one real, physical, position-localized mechanical catch. `FD 02` presence remained a reasonably good proxy for "did this specific command finish" throughout (it degraded exactly where the catch was), but the actual fix was mechanical, found only by a slow, deliberate hand-check of the specific failing range once statistical testing had localized it. If a similar pattern appears on Joint C or any future joint – clean single moves, degrading specifically under sustained one-direction travel – localize the range the same way (alternating vs. sustained statistical tests) before assuming it is a settings or protocol problem.

7.13 Next steps

- Run a broader confirmatory sweep across more of Joint B's full calibrated range (`home_position` $+12.87°$ to `opposite_limit` $-24.23°$), not just the $\sim$5–10° window tested so far, to rule out any other undiscovered catch elsewhere in the range.
- Re-verify reliability at a more practical production speed (testing here used a gentle `speed=15`; confirm the fix holds at `speed=50` and above before relying on it for real trajectories).
- `ENABLE_SHAFT_PROTECTION`'s false-positive behavior (Section 7.11) is still unexplained and remains off by default – this session's resolution did not depend on it and does not change that finding.
- **Joint C**: try `SR_CLOSE` + 1600 mA + confirm CANRSP, matching Joint B's verified config. Phase-resistance check and motor/driver swap test are still pending from 2026-08-25. If similar incomplete-move symptoms appear, run the same alternating-vs-sustained statistical comparison early rather than assuming an electrical cause.

8 Session Log: Large-Scale Motion, a Second Wall, and a Gear-Clearance Hypothesis (2026-08-27, continued)

With the $-2°$ to $-5°$ catch fixed and confirmed 100% reliable in both directions (Section 7.12), the user requested a much larger stress test: a full 360° rotation of the gearbox output, six times, in a bench configuration confirmed by the user to allow free/unlimited rotation (this specific setup does not have the joint's usual $\sim$37° calibrated envelope or any crossing cable at risk).

8.1 A real shaft-protection trip, and a lesson about tool-output reliability

The first full-360° attempt produced no visible movement, and its terminal output was lost to a tool error – a reminder that a lost or incomplete log means the actual hardware state must be re-verified from scratch (encoder position, enable state, protection state) before continuing, never assumed. Two subsequent attempts (a 30° command, then a 15° command) each tripped a **real** `ENABLE_SHAFT_PROTECTION` fault, visible directly on the driver's own physical display: `"Wrong Protect! Enter.."`. Unlike the earlier false-positive (Section 7.11), `0x88` appears to have stayed armed across the power-cycles performed earlier in the day (unlike `ENABLE_MOTOR`'s state, which

does not persist) and tripped for real partway into these longer moves. Pressing **Enter on the driver's own panel cleared the trip both times – no power-cycle was required**, updating the earlier assumption that a trip always needs a full power-cycle to clear. Explicitly sending `0x88 0x00` (disable) resolved the repeat trips.

8.2 The real behavior once protection was off: a stick-slip pattern, not a clean stall

With `ENABLE_SHAFT_PROTECTION` explicitly disabled, a 15° single command eventually completed (99.9% of commanded distance) but took a full **60 seconds** instead of the 1–2 seconds small moves take, moving in a distinct **stick-slip** pattern: long stalls of 10–30+ seconds at a fixed position, punctuated by sudden bursts (observed rates up to ∼5500 counts/s) that advanced several degrees at once, repeating 5–6 times before finishing.

8.3 Isolating the cause: removing the belt did not change anything

To test whether this was resistance in the belt-driven wrist mechanism downstream of the gearbox, the belt was physically removed and the identical 15° command re-run. **The stick-slip pattern was nearly identical** (same shape, similar total duration, if anything a longer single stall) with the belt off – despite the motor+gearbox now driving strictly less load than before. **This rules out the belt-driven wrist mechanism as the cause** and points at the motor/gearbox assembly itself.

8.4 Chunking into small commands did not avoid it either – it just found a different wall

Given every small isolated command earlier in the day was 100% reliable, the same 15° distance was tried as 15 separate 1° chunked commands (`joint_statistical_reliability_test.py`, sustained mode) rather than one large command. **Reps 1–6 were clean (99–101% each), then rep 7 was partial and reps 8–15 completely failed** (no `FD 02`, negligible motion) – a hard wall at a *different* absolute position (∼38–39° in this session's raw coordinate) than the originally-fixed −2° to −5° zone. Chunking did not solve the underlying problem; it simply relocated where the wall was encountered based on how far the sequence traveled before reaching it.

Caution: This looks like multiple distributed resistance points, not one isolated defect

Between the original catch, this new wall roughly 40° away, and a stick-slip pattern that persists across a 15° span independent of belt/load, the evidence now points at resistance distributed across the gearbox's own rotational range rather than a single fixable spot. The stick-slip signature (long stall, then sudden release) is also more consistent with tight/insufficient gear-tooth clearance (backlash) than with a single point of debris or damage – especially plausible for 3D-printed gears, where slight oversizing or ovality from the print process can cause mesh tightness to vary at different rotational positions.

8.5 Decision: reprint the gears with more clearance

Given the above, the user is decoupling the mechanism and reprinting the gears slightly smaller (more backlash) rather than continuing to locate and hand-work individual walls one at a time.

This is a well-motivated fix given the evidence – recommended refinements if pursued:

- Increase clearance modestly, not aggressively – too much backlash trades this problem for lost motion/imprecision in what is a precision joint.
- If the current gears show visible ovality or warping, consider print orientation/settings, not just uniform scale-down, since that would better explain why some rotational positions bind and others do not.
- Once reinstalled, re-verify across the **full range** using `joint_statistical_reliability_test.py` in both `alternating` and `sustained` modes, and at multiple starting positions – today's narrow-range tests gave false confidence more than once before a wider test surfaced the next issue.

8.6 Next steps

- Motor is disabled and the mechanism is being decoupled for the gear reprint. `can0` was brought down and `slcand` stopped per the standard shutdown procedure (Section **??**) before disconnecting the CANable.
- After reassembly with new gears, re-run the full bring-up procedure (Section 5) from Step 1 – do not assume the CAN ID, calibration, or any prior finding still applies to the rebuilt mechanism.
- If the stick-slip pattern persists even after the gear reprint, that would argue against the backlash hypothesis and toward the motor/driver itself (bearing, encoder mounting, or a firmware behavior specific to long single commands) – keep the belt-off isolation result and the chunked-vs-single-command comparison as reference points for that follow-up.

9 The Python Code Stack

To build an automated system, we separate our project into a clean library file (`mks_driver.py`) that handles the mathematical scaling transformations, and an execution test wrapper (`run_arm.py`).

9.1 The Core Driver: `mks_driver.py`

Caution: This listing is kept for historical reference – see Section 4 before using it

The `read_absolute_encoder()` method below sends opcode `0x30`, which `motor_types.hpp` does not define as a command at all (the real driver defines `READ_ENCODER = 0x31`); and the −813.0 pulses/degree figure below does not match the source-verified ≈3086.56 counts/degree for a 67.82:1-geared joint. Both are explained in Section 4. Every script added later in this document (Section 5) uses the corrected `0x31` opcode and does not use this pulses-per-degree constant.

Create a folder inside your workspace and save the following class module. This calculates our empirical real-world joint tracking scale of −813.0 **driver pulses per** 1° **of physical joint rotation**.

```python
import can
import time

class MKSServoDriver:
    def __init__(self, channel="can0", interface="socketcan"):
        self.bus = can.interface.Bus(channel=channel, interface=interface)
        self.ENCODER_CPR = 16384

        # Unified Joint Profile Configuration Matrix
        self.joint_profiles = {
            0x05: {"pulses_per_degree": -813.0, "invert": False}, # Joint B
        }

    def checksum(self, motor_id, data_bytes):
        """MKS SUM-8 Checksum validation."""
        return (motor_id + sum(data_bytes)) & 0xFF

    def send_frame(self, motor_id, data_bytes):
        crc = self.checksum(motor_id, data_bytes)
        frame = data_bytes + [crc]
        msg = can.Message(arbitration_id=motor_id, data=frame, is_extended_id=False)
        self.bus.send(msg)
        return frame

    def set_motor_state(self, motor_id, enable=True):
        state_byte = 0x01 if enable else 0x00
        self.send_frame(motor_id, [0xF3, state_byte])
        time.sleep(0.1)

    def read_absolute_encoder(self, motor_id, timeout=1.0):
        self.send_frame(motor_id, [0x30])
        end = time.time() + timeout
        while time.time() < end:
            resp = self.bus.recv(timeout=end - time.time())
            if resp is not None and resp.arbitration_id == motor_id and resp.data[0] == 0x30:
                carry = int.from_bytes(resp.data[1:5], byteorder="big", signed=True)
                value = int.from_bytes(resp.data[5:7], byteorder="big", signed=False)
                return (carry * self.ENCODER_CPR) + value
        raise TimeoutError(f"Motor {motor_id} failed to return encoder data.")

    def move_relative_degrees(self, motor_id, degrees, speed=25, accel=5):
        current_encoder = self.read_absolute_encoder(motor_id)
        profile = self.joint_profiles[motor_id]

        target_pulses = int(degrees * profile["pulses_per_degree"])

        # Absolute Software Boundary Filters (Safety Workspace Cage)
        estimated_final_encoder = current_encoder + int(target_pulses * -0.947)
        MIN_LIMIT, MAX_LIMIT = -73000, 73000
        if estimated_final_encoder < MIN_LIMIT or estimated_final_encoder > MAX_LIMIT:
            print(f"Movement Blocked: Target {estimated_final_encoder} violates safety limits!")
            return

```

```
        pos = target_pulses & 0xFFFFFF
        data = [0xF4, (speed >> 8) & 0xFF, speed & 0xFF, accel, (pos >> 16) & 0xFF
    , (pos >> 8) & 0xFF, pos & 0xFF]
        self.send_frame(motor_id, data)

    def close(self):
        self.bus.shutdown()
```

Listing 9: mks_driver.py Module

9.2 The Target Runner: `run_arm.py`

This basic executable test sequence leverages our driver module to run a clean, safe, relative shift:

```
#!/usr/bin/env python3
from arctos_can.mks_driver import MKSServoDriver
import time

JOINT_B_ID = 0x05
arm = MKSServoDriver(channel="can0")

try:
    start_pos = arm.read_absolute_encoder(JOINT_B_ID)
    print(f"Current Position Baseline: {start_pos} encoder counts")

    print("Powering up motor coils...")
    arm.set_motor_state(JOINT_B_ID, enable=True)

    TARGET_ANGLE = 5.0
    print(f"Executing a large continuous sweep of +{TARGET_ANGLE} degrees...")
    arm.move_relative_degrees(JOINT_B_ID, degrees=TARGET_ANGLE, speed=60, accel
    =20)

    time.sleep(3.0) # Allow mechanism transit time settling
    end_pos = arm.read_absolute_encoder(JOINT_B_ID)
    print(f"End encoder: {end_pos} (Delta: {end_pos - start_pos})")

finally:
    print("Shutting down cleanly.")
    arm.set_motor_state(JOINT_B_ID, enable=False)
    arm.close()
```

Listing 10: run_arm.py Execution Script

9.3 Compiling and Launching over the ROS 2 Command Line

Make sure your workspace is built, sourced, and execute standard command-line tools to monitor parameters:

```
# 1. Compile your localized package source workspace
cd ~/arctos_ws
colcon build --packages-select arctos_can_control
source install/setup.bash

# 2. Run the main processing node thread
ros2 run arctos_can_control joint_control

# 3. Open a separate terminal tab to audit live incoming SI Radians telemetry
```

```
ros2 topic echo /joint_states

# 4. Open a third tab to inject a precise motion command into the active network
    stream
ros2 topic pub --once /joint_b_command std_msgs/msg/Float64 "{data: 5.0}"
```

Listing 11: Building and Testing over ROS 2 CLI